CLASE / INTELIGENCIA ARTIFICIAL

OpenSpec

Del requerimiento al código, sin dejar decisiones al azar.

Desarrollo dirigido por especificaciones (SDD)



OBJETIVOS

La ruta de esta clase

Cinco preguntas que podrás responder al terminar.

01 ¿Qué es SDD?

02 ¿Por qué no basta un prompt?

03 ¿Cómo funciona OpenSpec?

04 ¿Qué archivos y comandos usa?

05 ¿Cómo validar una implementación?

CONCEPTO

SDD: primero acordamos qué construir

OpenSpec convierte acuerdos en artefactos revisables y versionables.

SDD

Especificar → diseñar → planificar → implementar → comprobar.

OpenSpec

Organiza requisitos, decisiones y tareas para trabajar con asistentes de IA.

La especificación orienta; las pruebas verifican.

COMPARACIÓN

No es lo mismo pedir que especificar

La diferencia es quién decide las reglas del producto.

Prompt suelto

“Haz un login seguro”.

La IA interpreta alcance, errores y reglas de negocio.

Con OpenSpec

“Implementa el login siguiendo estos escenarios y decisiones”.

El equipo controla qué se acepta.

PROBLEMAS

Tres costos de improvisar

Especificar temprano permite revisar antes de programar.

Ambigüedad

“Funciona” significa cosas distintas para cada persona.

Deuda técnica

Decisiones implícitas producen soluciones inconsistentes.

Revisión cara

Corregir requisitos después del código cuesta más.

OpenSpec ayuda a reducir estos riesgos, pero no reemplaza el criterio de ingeniería.

FLUJO

Del cambio propuesto a la funcionalidad

Flujo habitual de trabajo con OpenSpec.



Explorar y sincronizar son opcionales. La especificación puede evolucionar.

ARTEFACTOS

Cuatro documentos, cuatro preguntas

Ejemplo: openspec/changes/add-login/

proposal.md

¿Por qué y qué cambia?

spec.md

¿Qué comportamiento se espera?

design.md

¿Cómo se implementará?

tasks.md

¿Qué trabajo concreto sigue?

FASE 1 / PROPOSAL.MD

Antes de construir, definimos el alcance

Caso de estudio: agregar autenticación a una aplicación.

```
# proposal.md

Problema: el sistema no tiene login.

Objetivo: acceso por correo y contraseña.

Incluye: login, errores y cierre de sesión.

Excluye: registro y recuperación de cuenta.
```

PREGUNTA CLAVE

¿Estamos resolviendo el problema correcto?

FASE 2 / SPEC.MD

Describir resultados, no adivinar intenciones

Los escenarios transforman expectativas en condiciones verificables.

```
## Requirement: Login con credenciales

El sistema DEBE autenticar usuarios activos.


### Scenario: credenciales válidas

WHEN envía correo y contraseña correctos

THEN recibe una sesión válida.
```

OTRO ESCENARIO

Contraseña errónea

Se informa el error sin exponer datos sensibles.

FASE 3 / DESIGN.MD

Decidir cómo funcionará

La IA no debería inventar decisiones de arquitectura sin revisión.

API	POST /auth/login
Aplicación	Servicio de autenticación
Seguridad	Hash, sesión y límites de intentos
Observabilidad	Logs sin credenciales

FASE 4 / TASKS.MD

Convertir el diseño en tareas verificables

/opsx:apply ejecuta el plan; la revisión de calidad sigue siendo necesaria.

- [ ] Definir contrato de POST /auth/login
- [ ] Validar formatos y entradas
- [ ] Implementar autenticación
- [ ] Controlar credenciales incorrectas
- [ ] Probar éxito, fallo y casos límite
- [ ] Comparar código final con la especificación

HERRAMIENTAS

Comandos de OpenSpec

Diferenciar instrucciones del agente y operaciones de terminal.

EN EL CHAT DEL AGENTE

```
/opsx:explore
/opsx:propose add-login
/opsx:apply
/opsx:sync
/opsx:archive
```

EN TERMINAL

```
npm install -g @fission-ai/openspec@latest
openspec init
openspec list
openspec status --change add-login
```

CONTROL DE CALIDAD

No aceptar código solo porque compila

Cinco preguntas para revisar la salida del agente.

✓	**Alcance**	¿Implementó solo lo acordado?
✓	**Requisitos**	¿Se cumplen todos los escenarios?
✓	**Diseño**	¿Respetó las decisiones técnicas?
✓	**Pruebas**	¿Cubrió errores y casos límite?
✓	**Seguridad**	¿Protege credenciales y sesiones?

ACTIVIDAD / 100 PUNTOS

Reto: carrito de compras

En equipos: añadir y eliminar productos, usando el flujo de OpenSpec.

Entregables

proposal.md • alcance
spec.md • 3 escenarios
design.md • decisiones
tasks.md • mínimo 5 tareas
Revisión antes de /opsx:apply

RÚBRICA

Claridad de requisitos	30
Escenarios verificables	25
Diseño justificado	20
Tareas ejecutables	15
Revisión crítica	10

CIERRE

La IA puede escribir código rápido.

OpenSpec ayuda a decidir qué código merece escribirse.

ESPECIFICAR | REVISAR | IMPLEMENTAR | PROBAR | ARCHIVAR

github.com/Fission-AI/OpenSpec